BUILD SPECIFICATION · v2 · 2026

Polymarket BTC 5-Minute Bot

Comprehensive in-and-out specification for the development process.
Decisions, architecture, phases, gates, costs, discipline.

OWNER
Hamad

STARTED
2026-05-04

OPERATOR JURISDICTION
UAE (permitted)

STAGE
pre-Phase 0

Table of Contents

What you'll find in this document

01	**Executive Summary**	Thesis, edge ranking, win-rate target.	p. 3
02	**Locked Decisions**	All 18 locked decisions with rationale.	p. 4
03	**Tooling Inventory — Claude**	Skills, MCPs, slash commands, plugins.	p. 5
04	**Tooling Inventory — Stack**	Engine, signing, data feeds, deps.	p. 6
05	**Tooling Inventory — Infra**	VPS, monitoring, secrets, CI.	p. 7
06	**Architecture — Runtime**	Three concurrent loops on the VPS.	p. 8
07	**Architecture — Ensemble**	3 agents + meta + Tier-3 fallback.	p. 9
08	**Architecture — Wallets**	Funder/Signer separation, sweeps.	p. 10
09	**Phase 0 — Foundations**	Project skeleton, accounts, wiring.	p. 11
10	**Phase 1 — Wallet & Auth**	Derive creds, test sign, then fund.	p. 12
11	**Phase 2 — Data Plane**	Spot, oracle, market feeds; thesis verify.	p. 13
12	**Phase 3 — Backtest + A/B**	Deterministic vs LLM ensemble.	p. 14
13	**Phase 4 — Paper Trading**	14 days live data, regime coverage.	p. 15
14	**Phase 5 — VPS Provisioning**	Dublin Lightsail, hardening, staging.	p. 16
15	**Phase 6 — Live Trading**	5 kill-switches, $5 flat sizing.	p. 17
16	**Phase 7 + 8**	Scale, hardening, ongoing.	p. 18
17	**Risk Register**	Severity-tagged risks + mitigations.	p. 19
18	**Cost & Timeline**	Cumulative spend per phase.	p. 20
19	**Operating Discipline**	25 anti-drift rules pin-above-desk.	p. 21
20	**Sources & References**	Verified canonical links.	p. 23

Executive Summary

The thesis and the numbers

Polymarket runs continuous 5-minute binary markets on whether BTC closes higher or lower than its starting price. Markets are deterministic with windows aligned to **floor(now/300)*300**. Resolution = the Chainlink BTC/USD snapshot at **window_ts + 300** on Polygon. Chainlink BTC/USD on Polygon is reported as a **60s heartbeat with no deviation threshold** (pending on-chain verification in Phase 2) — that gap, combined with thin orderbooks ($5-50K/window) and the empirical fact that ~15-20% of windows are decided in the final 10 seconds, is the exploitable inefficiency.

60s
Chainlink heartbeat (pending verify)

1-5ms
Dublin VPS to CLOB

$1k
Starting bankroll · $5 flat sizing

≥3pp
LLM uplift vs baseline to keep agents

Edge sources (ranked)

Oracle latency	60s heartbeat, no deviation; spot moves while oracle is silent
Window-delta dominance	single highest-weight signal — bigger than RSI/EMA combined
Last-10s dynamics	~15-20% of windows decided here; tokens still sub-$0.95
Maker rebates	100% of taker fees redistributed daily under CLOB v2
Cross-venue spot arb	Binance/Coinbase WS ahead of Polymarket UI by 100ms+

What's NOT the edge

- TA on 5-min timeframe — RSI/EMA on 1m candles is mostly noise.
- Manual trading — humans cannot react fast enough at T-10s.
- Deep-volume markets — the inefficiency lives in thin books.
- Raw round-trip latency — co-located bots in eu-west-2 beat us; our edge is signal quality + last-second timing.

Cost-to-first-live-dollar: ~30 calendar days from Phase 0; ~$170 setup spend (VPS + APIs + RPC + minor tools); $1,000 working capital that is at risk, not consumed. Sustained ops once live: ~$200/mo budget driven primarily by Anthropic API (empirically $0.19-0.58/hour on a similar bot the operator already runs).

Locked Decisions

Do not relitigate without explicit go

#	Decision	Rationale
01	UAE-resident operator	Polymarket-permitted jurisdiction
02	Dublin AWS Lightsail eu-west-1	Sub-5ms to Polymarket CLOB; avoids UK geo-block
03	NautilusTrader engine + py-clob-client	Production-grade; official Polymarket adapter
04	Anthropic API only — no OpenAI fallback	Simpler matrix; deterministic Tier-3 catches LLM failures
05	3 agents + meta-decider	Orderbook, oracle-gap, momentum, weighted by Sonnet meta
06	Tier-3 = window-delta + oracle-gap, threshold ≥ 0.6	Fires on strong signal; never skip a window
07	Telegram bot via BotFather for alerts	Reliable global push; supports inbound /halt
08	GitHub private repo	Default code host; CI via GitHub Actions
09	$5 flat per trade for first 100 trades	Pre-Kelly learning budget; clears $4.75 min; 0.5% sizing at $1k bankroll
10	$1,000 starting bankroll	Real capital; HW wallet trigger ($2k) one rung away; meaningful Kelly transition
11	Funder/Signer wallet separation from Phase 1	Funder = working capital; Signer ≤ 2 days bankroll
12	Hardware wallet at $2k bankroll trigger	One rung past $1k start; first scaling step ($3k) triggers purchase
13	Phase 1 order: derive creds → test sign → fund	Don't strand $20 on an unverifiable wallet
14	Anthropic budget: $50 → $150 (Phase 4) → $300 (Phase 6)	Empirical $0.19-0.58/hr; sustained $200/mo OPEX
15	UTC everywhere — logs, DB, daily notes	Mixing TZ + window timestamps is a future bug
16	LLM A/B in Phase 3; keep only if uplift ≥ 3pp	LLM ensemble retained only if it earns its cost
17	Polymarket gasless relayer for all order submission	Closes the public-mempool MEV attack surface
18	Obsidian vault on laptop only; bot writes to git state/	Single source of truth; no VPS Obsidian plumbing

Tooling Inventory — Claude side

Every skill, MCP, slash command, plugin

Claude skills

Item	Source / install	Where	Phase
agent-skills (Polymarket official)	`git clone github.com/Polymarket/agent-skills`	~/.claude/skills/agent-skills/	0 (clone), 1+ (use)
anthropic-skills:pdf	built-in	Skill tool	as needed
anthropic-skills:docx	built-in	Skill tool	as needed
custom: poly-bot-skills	write in Phase 0	~/.claude/skills/poly-bot/	0 onward

MCP servers

MCP	Purpose	Where	Phase
iansinnott/obsidian-claude-code-mcp	Read/write the laptop vault under ./state/	Obsidian plugin	0+
caiovicentino/polymarket-mcp-server	45 tools, real-time WS, exposure caps (research only)	stdio, ~/.mcp/	2+
artvandelay/polymarket-agents	10 tools: odds, orderbook, spread, history	stdio	2+
agent-next/polymarket-paper-trader	Paper-trading sim from chat	stdio	3-4
github (gh CLI)	Read repos, file issues, manage PRs	Bash	0+

Slash commands & plugins

Command / plugin	When	Phase
/init	Seed project CLAUDE.md once layout is on disk	0
/security-review	Before each phase touching signing or fund movement	1, 5, 6
/loop	Periodic check (e.g. monitor backtest) — sparing use	3-4
/schedule	Recurring agent tasks — only after Phase 6	7+
/clear	After drift detected mid-conversation	any
design suite (skills:design-critique)	Optional UI / dashboard work	later

Tooling Inventory — Stack

Engine, signing, data feeds, deps

Core runtime

Layer	Choice	Repo / package	Phase
Language	Python 3.11+	system + uv	0
Package manager	uv	`astral.sh/uv`	0
Trading engine	NautilusTrader	github.com/nautechsystems/nautilus_trader	3+
Order client	py-clob-client	github.com/Polymarket/py-clob-client	1+
Backtest scaffold	fork as needed	github.com/evan-kolberg/prediction-market-backtesting	3
Type check	mypy --strict	uv add --dev mypy	0+
Lint + format	ruff	uv add --dev ruff	0+
Test framework	pytest + pytest-asyncio	uv add --dev pytest pytest-asyncio	0+
Import boundaries	import-linter	uv add --dev import-linter	0+

Data plane

Source	Endpoint / library	Use	Phase
Binance WS	wss://stream.binance.com:9443/ws/btcusdt@trade	Spot ticks (primary)	2+
Coinbase WS	wss://advanced-trade-ws.coinbase.com	Spot ticks (redundant)	2+
Polygon RPC	Alchemy free tier (default)	Chainlink AnswerUpdated subscription	2+
Polymarket WS	wss://ws-subscriptions-clob.polymarket.com/ws/market	Active 5m market orderbook + trades	2+
Storage	SQLite (trades.db) + Parquet (ticks/)	Local, durable, no external deps	2+

Reference repos (read-only mining)

Repo	Why we cloned it	Status
Polymarket/py-clob-client	Auth flow, signing primitives	clone
Polymarket/agents	Patterns for autonomous agent design	mine
Polymarket/poly-market-maker	Reference market-maker keeper	mine
evan-kolberg/prediction-market-backtesting	Polymarket adapter for NautilusTrader BT	fork
aulekator/Polymarket-BTC-15-Minute-Trading-Bot	Production-grade 15m bot architecture	mine
ThinkEnigmatic/polymarket-bot-arena	Multi-strategy arena pattern	mine
Archetapp gist (5m bot)	Best timing/window logic in the wild	mine

Tooling Inventory — Infra

VPS, monitoring, secrets, CI, ops

Hosting + ops

Layer	Choice	Why
VPS	AWS Lightsail eu-west-1 Dublin	Sub-5ms to CLOB; ~$5/mo
OS	Ubuntu 24.04 LTS	Long-term stability, security updates
Process supervisor	systemd	Auto-restart, journalctl logs
Time discipline	chrony	Clock skew < 10ms enforced
Inter-process state	redis	Coordinate kill-switches, alert queue
DB replication	litestream	trades.db → S3-compatible offsite

Monitoring + alerts

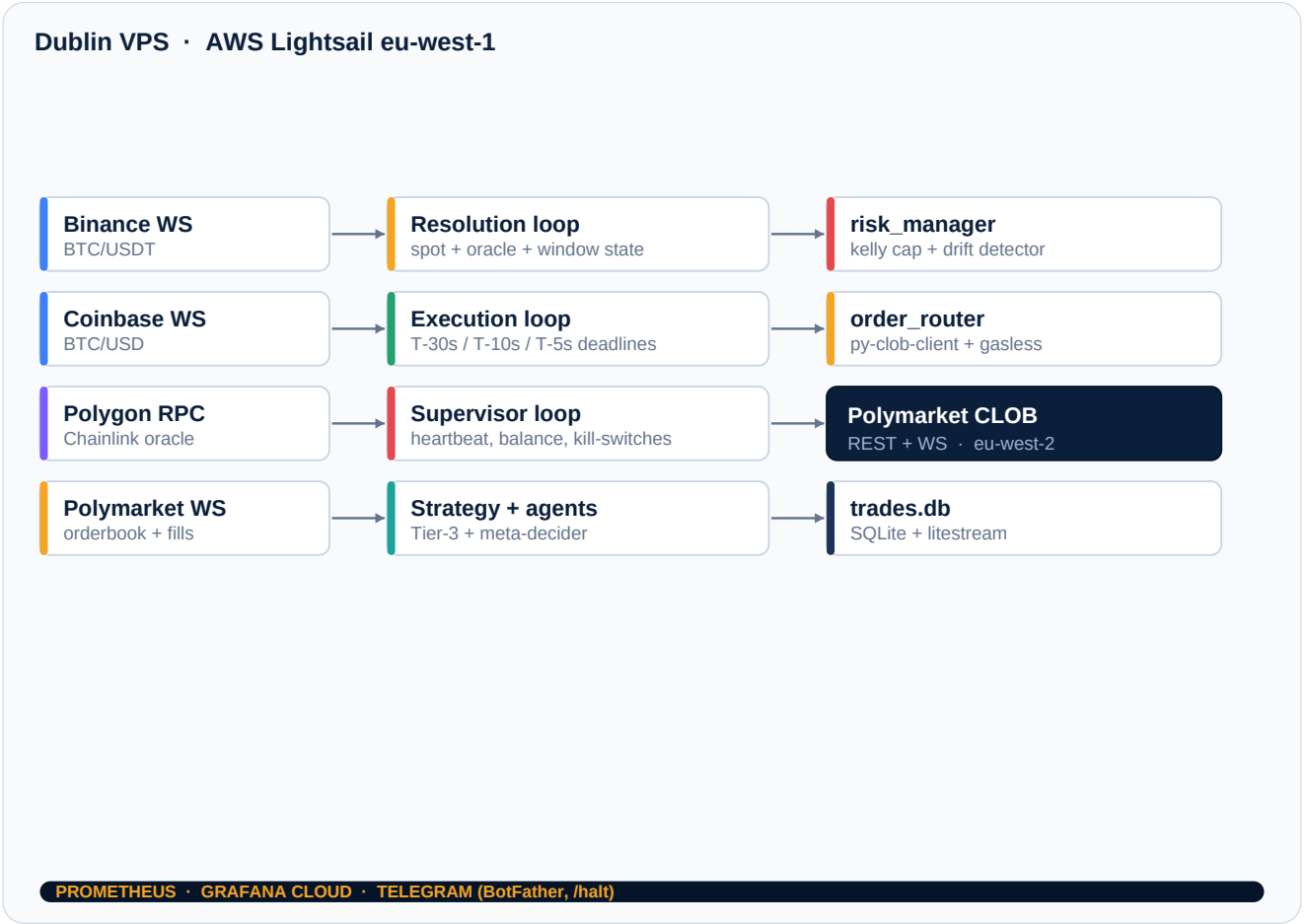
Tool	Use
prometheus-node-exporter	System metrics on the VPS (CPU, RAM, disk, net)
Bot Prometheus exporter	Decision counts, win-rate, fill latency, balance
Grafana Cloud free tier	Dashboards, 14d retention; warn/info
Telegram bot (BotFather)	CRITICAL alerts; inbound /halt; mobile push
Backblaze B2 (default OD-2)	Offsite backup of trades.db via litestream; ~$0.005/GB/mo

Secrets, CI, deployment

Tool	Use
direnv + .envrc	Project-scoped env vars; .envrc gitignored
age	Encrypted backup of wallet keys; offline-only
1Password CLI (or pass)	Secrets injection at runtime via op run --
GitHub Actions	CI on push: lint, type, test, header-check, TODO-grep, import-linter
pre-commit	Local enforcement of the same checks before push
Deployment	git pull + systemctl restart (no second VPS until $1k bankroll)

Architecture — Runtime (three loops)

Three concurrent loops on the Dublin VPS. Resolution loop continuously knows *'what does the active market resolve as if it ended right now.'* Execution loop fires orders by deadline at T-30s / T-10s / T-5s, never skipping a window. Supervisor loop runs every minute checking heartbeat, balance reconciliation, kill-switch state.



Laptop · Obsidian + Claude Code

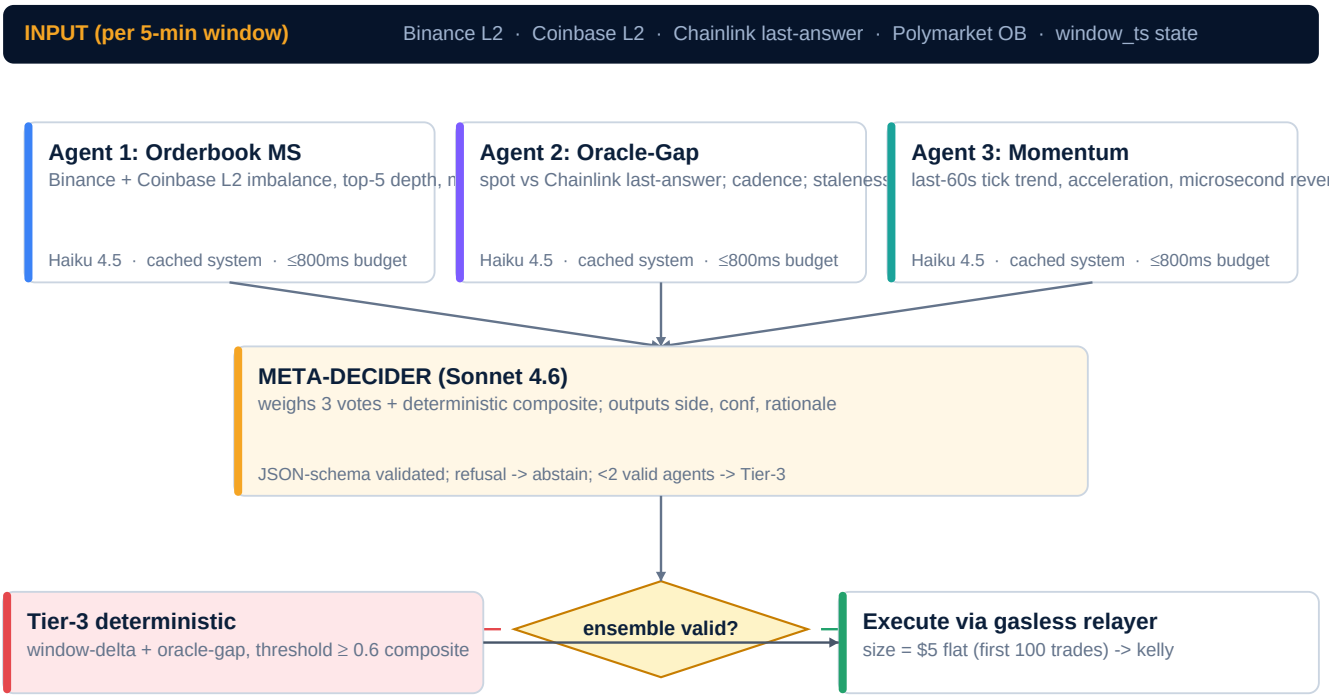
Vault indexes ./state/ ← bot writes runbooks/postmortems via git push

Read/review/decide. No live trading from laptop. Funder key never on VPS.

Architecture — Agent Ensemble + Tier-3

3 Anthropic agents, 1 meta-decider, deterministic fallback

Each window: 3 specialized Haiku 4.5 agents return JSON-validated votes (side, confidence, rationale). Meta-decider Sonnet 4.6 weighs the 3 votes plus the deterministic composite signal. Schema-fail or refusal → that agent abstains. If fewer than 2 valid votes, the deterministic Tier-3 fires standalone with a raised confidence threshold (composite ≥ 0.6). Tier-3 also catches API rate-limits, latency budget breaches, and any case where the LLM stack would skip a window.

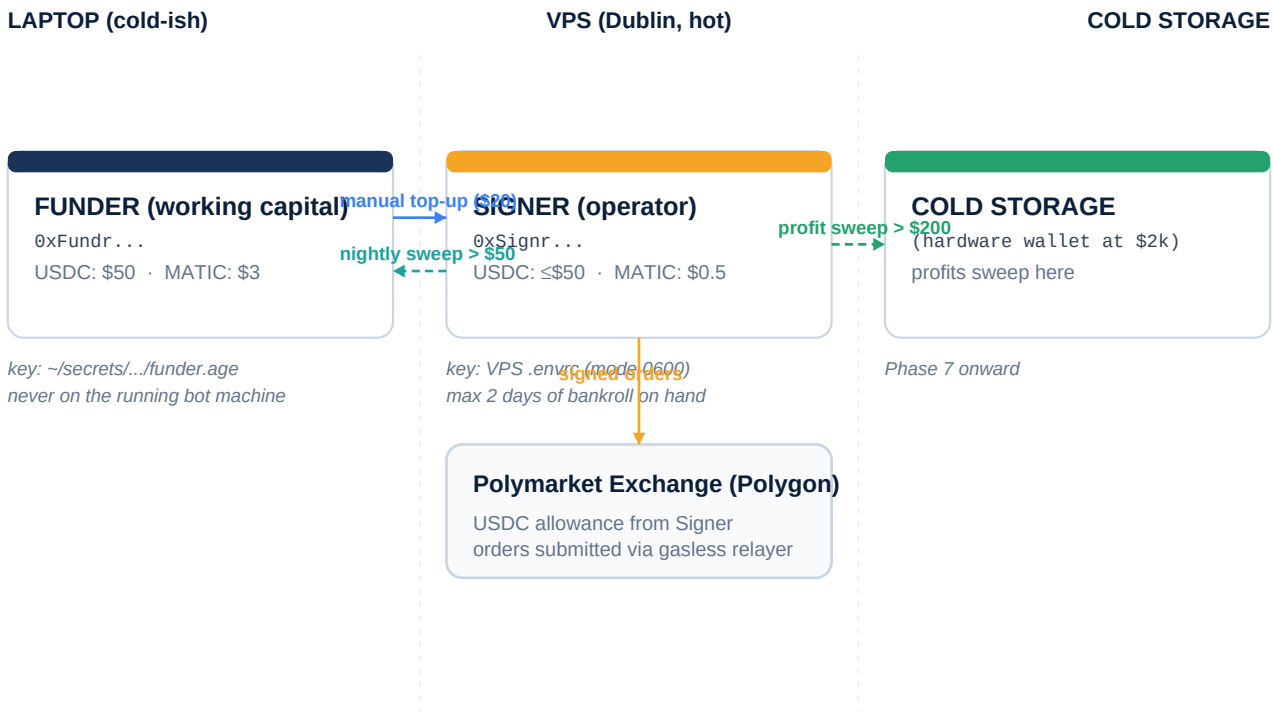


Tier-3 fires when: any agent 429s, latency budget at T-15s blown, schema-fail, or <2 valid votes. Never skip a window.

Architecture — Wallet Topology

Funder / Signer / Cold; sweep flows

Three wallet zones with explicit sweep policy. Funder lives on the laptop (age-encrypted key, never on the VPS). Signer lives on the VPS (mode-0600 .envrc, max 2 days of bankroll on hand). Cold storage starts at $2k bankroll (hardware wallet, Phase 7). Profit cascade: Signer balance > $50 sweeps nightly to Funder; Funder > $200 sweeps to Cold.



If Signer key leaks: max loss = USDC currently on Signer (≤2 days bankroll). Funder + Cold are unaffected.

PHASE 0 · Foundations 1 day · $0

Intent: Stand up the dev environment, verify all account prerequisites, seed the project skeleton. No money, no keys.

Tasks

- Initialize project skeleton with uv (Python 3.11, mypy --strict, ruff, pytest, import-linter, pre-commit).
- Init git, create GitHub private repo (gh repo create), push CI workflow with TODO-grep + header-check.
- Create Telegram bot via BotFather; capture token + chat_id; smoke-test sendMessage from CLI.
- Drop Polymarket/agent-skills into ~/.claude/skills/; verify Claude can answer auth questions citing skill.
- Clone reference repos into references/ (gitignored): py-clob-client, agents, poly-market-maker, etc.
- Wire Obsidian to vault rooted at ./state/; install obsidian-claude-code-mcp plugin.
- Run /init to seed CLAUDE.md with locked decisions + anti-vibe rules + RUNBOOK pointer.

GATE bash runbooks/gates/phase-0.sh exits 0 (mypy/ruff/test green; remote exists; agent-skills present; Telegram smoke received).

Rollback: rm -rf and start over; nothing of value lost.

PHASE 1 · Wallet & Auth 0.5 day · ~$5 gas

Intent: Generate Funder + Signer wallet pair. Derive L2 creds. Prove signing works BEFORE funding. Then $5 gasless test.

Tasks

- Resolve OD-4: read agent-skills/gasless.md; confirm relayer flow; document in design/gasless.md.
- Generate Funder + Signer offline; age-encrypt to ~/secrets/polymarket-5m/.
- Derive Polymarket L2 creds from Signer privkey via py-clob-client (no funds needed).
- Run scripts/test_auth.py — signed read-only call against /info, returns Signer address.
- ONLY THEN: fund Funder $50 USDC + $5 MATIC; sweep $20 USDC + 0.5 MATIC to Signer.
- Approve Polymarket exchange contract for Signer USDC (one-time on-chain tx).
- Place + cancel a $5 limit order on a non-time-critical market via gasless relayer.

GATE bash runbooks/gates/phase-1.sh exits 0 (auth test green, $20 funded, allowance set, $5 order placed and canceled, evidence captured

Rollback: If auth test fails before funding: debug .envrc. If order stuck: wait 5 min, then contact Polymarket support.

PHASE 2 · Read-only Data Plane

2-3 days dev + 7 days capture · ~$5 RPC

Intent: Three reliable feeds (spot, oracle, market). 7 days of clean capture. Verify the Chainlink heartbeat thesis.

Tasks

- Resolve OD-1: choose Polygon RPC (default Alchemy free tier); get WSS URL into .envrc.
- Design data plane: Feed protocol, per-source files, Parquet writer per UTC-day.
- Implement spot_feed.py (Binance + Coinbase WS, fan-in aggregator).
- Implement oracle_watcher.py (Chainlink BTC/USD AnswerUpdated subscription on Polygon).
- Implement market_feed.py (Polymarket WS for active 5m BTC market).
- Implement resolver.py: hypothetical_now() vs actual_resolution(window_ts).
- Run continuously for 7 calendar days under tmux/launchd; daily file-growth check.
- Run scripts/analyze_oracle.py over the 7-day capture; verify median heartbeat ∈ [55s, 65s] and zero sub-30s deviation triggers.

GATE bash runbooks/gates/phase-2.sh exits 0 (mypy/ruff/test green, 7 daily ticks files exist, oracle analysis criteria all pass).

Rollback: Feeds drop overnight: investigate cause (WS provider hiccup), restart, EXTEND capture window. Do not advance with gaps.

PHASE 3 · Backtest + LLM A/B Test

3-4 days · $15-50 (with cache discipline)

Intent: Build deterministic baseline. Run on 21 days walk-forward. Test if LLM ensemble adds ≥3pp uplift. If not, ship deterministic alone.

Tasks

- Confirm Anthropic credits ≥ $50 + cost-tracker hard-kill at $35/pass.
- Wire NautilusTrader Polymarket adapter (or fork evan-kolberg's BT scaffold) to captured Parquet.
- Implement deterministic strategy: composite = w_wd*window_delta + w_og*oracle_gap; fire if |composite| ≥ 0.6 AND TTL ∈ [10,30].
- Implement fill simulator: top-5 orderbook history + self-impact model (calibrate later from real fills).
- Walk-forward: rolling 7-day train, 1-day test, 30 days total (after 7-day warmup).
- Implement 3 agents (orderbook, oracle-gap, momentum) + Sonnet meta-decider; explicit prompt-cache layout.
- Run A/B: deterministic-only vs deterministic+ensemble; report win-rate, latency, $/decision.
- Resolve OD-5: keep ensemble iff uplift ≥ 3pp AND p99 latency ≤ 4s AND $/decision ≤ $0.01.

GATE bash runbooks/gates/phase-3.sh exits 0 (≥2,500 trades, deterministic ≥ 53%, A/B verdict file committed, total spend ≤ $50).

Rollback: Deterministic < 53%: don't introduce LLMs. Iterate weights, fee model, threshold. Extend capture if regime-biased.

PHASE 4 · Paper Trading on Live Venue

14 calendar days · $50-195 (top up Anthropic to $150 first)

Intent: Run chosen strategy in paper mode against live data for 14 days. Validate the simulator. Catch every bug backtest didn't.

Tasks

- Top up Anthropic to $150 (covers 14d × 24/7 at empirical $0.19-0.58/hr upper bound).
- Implement _paper/shadow_book.py — receives strategy decisions, simulates fills against live orderbook.
- Implement schema validation + refusal handling: invalid response → agent abstains; <2 valid → Tier-3.
- Validate simulator: 100 hypothetical decisions over 24h; ≥90% within 1¢ of real top-of-book.
- Plan start date so 14d covers ≥1 CPI/FOMC, ≥1 weekend, ≥1 BTC daily move > 2%; extend if not covered.
- Pre-trade decision logging: full input snapshot per decision into state/decisions/<date>.jsonl.
- Daily ritual: review previous day's PnL, anomalies, simulator divergence vs backtest > 20%.

GATE bash runbooks/gates/phase-4.sh exits 0 (14 days clean logs, regime coverage met, simulator ≥90%, paper PnL within 20% of backtest

Rollback: Paper diverges > 20%: debug simulator, NOT strategy. Extend if cause unclear.

PHASE 5 · VPS Provisioning

0.5 day setup + 24h staging · $5/mo VPS

Intent: Move execution to Dublin. Verify WS-specific latency. Stand up monitoring + Telegram. Stage 24h before going live.

Tasks

- Provision Lightsail eu-west-1 Dublin: 2vCPU/4GB/80GB NVMe, Ubuntu 24.04, static IP.
- Harden: SSH key-only on port 2222, ufw, fail2ban, unattended-upgrades, non-root user, chrony.
- Install: Python 3.11, uv, redis, prometheus-node-exporter, litestream.
- Latency-check WS specifically: wscat round-trip to wss://ws-subscriptions-clob.polymarket.com/ws/market — p50 < 5ms, p99 < 15ms.
- Create two systemd units: bot-staging (paper, port 9101) and bot (live, port 9100, disabled).
- Wire Prometheus → Grafana Cloud; tg-alerter.service for Telegram CRITICAL alerts; /halt inbound test.
- Move Signer key via age-encrypted SSH transfer; .envrc mode 0600. Funder key NEVER touches VPS.
- Run bot-staging for 24h watching metrics + alerts.

GATE bash runbooks/gates/phase-5.sh exits 0 (staging healthy 24h, WS p99 < 15ms, chrony offset < 10ms, /halt test passed, litestream repli

Rollback: Latency > 15ms: try a different AZ within eu-west-1. /halt fails: debug tg-alerter polling.

PHASE 6 · Live with Kill-Switches

14 calendar days · $1,000 capital + ~$50-195 LLM

Intent: $1,000 funded. $5 flat per trade (0.5% sizing). Five kill-switches all tested. Postmortem any anomaly > $3.

Tasks

- Top up Anthropic to $300 (covers Phase 6 14d × 24/7 + buffer).
- Implement 5 kill-switches: heartbeat, slippage, drawdown -$50/-$100, reconciliation drift, win-rate drift.
- Synthetic-trip every kill-switch in test before live: tests/kill_switches/*.
- Strategy sizing.py: $5 flat for first 100 trades; then max($5, kelly * bankroll * conf), cap 10% = $100.
- Funder/Signer split: Funder $900, Signer ≤$100; nightly sweep Signer > $100 back to Funder.
- Daily 12:00 UTC ritual: scripts/daily_recon.py → Telegram + state/daily/.
- Postmortem any kill-switch trip OR > $3 unexplained loss in state/postmortems/.
- Optional after 7 stable days: maker-rebate layer (cap 25% of bankroll in resting orders).

GATE bash runbooks/gates/phase-6.sh exits 0 (14d live, ≥100 trades, all 5 kill-switches synthetic-tripped, no recon drift > $0.50, no daily brea

Rollback: Drawdown breaker trips: 24h cooling-off MANDATORY. Postmortem signed off by user before re-enable.

PHASE 7 · Scale + Maker Rebates

ongoing — 7d per rung minimum · bankroll-driven

Intent: Step bankroll only after positive 7-day run on previous rung. $1k → $3k → $10k → $30k. Hardware wallet at the $3k step.

Tasks

- Rung 7.1 — $3,000: Kelly × 0.25; resolve OD-6 (Ledger for Funder, ~$80); breaker -$150/day.
- Rung 7.2 — $10,000: Kelly × 0.20; multi-account proxy wallets; per-account caps; breaker -$500/day.
- Rung 7.3 — $30,000: Kelly × 0.15; LLC consideration; Koinly tax integration; breaker -$1,500/day.
- Variant arena: second strategy in shadow; weekly compete-and-allocate (pattern from polymarket-bot-arena).
- Quarterly: scripts/refit_weights.py over last 30d; promote only if uplift ≥ 1pp AND 48h paper validation matches.

GATE Per-rung: 7+ days positive PnL, zero kill-switch trips, no unexplained losses on prior rung.

Rollback: Step back one rung; never skip rungs.

PHASE 8 · Hardening

ongoing · TBD

Intent: Production-grade ops. Migrations. Standby. Multi-account. Re-evaluate dropped LLM ensemble if deterministic decays.

Tasks

- Track USDC.e → native USDC migration; double-write/read during rollout.
- Frankfurt warm standby Lightsail; litestream replication; failover playbook.
- Multi-account proxy wallets at > $50k; per-account caps and daily reconciliation.
- Quarterly Polymarket changelog review (1 day/quarter budgeted).
- Re-evaluate dropped LLM ensemble if deterministic strategy decays > 3pp.
- Re-evaluate OpenAI fallback once Tier 4 Anthropic limits are insufficient.

GATE Each item has its own design doc + implementation + test pass.

Rollback: Items can be deferred; none block live ops.

Risk Register

Severity-tagged. Read at every phase boundary.

#	Severity	Risk	Mitigation
01	BLOCKER	**Jurisdiction (resolved: UAE permitted)** Polymarket geofences US/UK/FR/BE/SG/TH/AU/Ontario.	UAE confirmed permitted. If user moves jurisdictions, project may need to halt.
02	BLOCKER	**MEV / public-mempool front-running** Public RPC orders can be sandwiched by Polygon MEV bots.	Use Polymarket gasless relayer for ALL order submission (locked decision #17). Never public-RPC orders.
03	BLOCKER	**Obsidian-on-VPS architectural mismatch** Obsidian is desktop GUI; can't run on the bot VPS.	Vault stays on laptop; bot writes markdown to ./state/ in git, laptop pulls (locked decision #18).
04	MAJOR	**Wallet compromise / single key blast radius** Leaked private key drains everything immediately.	Funder/Signer separation. Signer ≤2 days bankroll. Hardware wallet for Funder at $2k.
05	MAJOR	**Strategy edge defensibility** Co-located bots in eu-west-2 may beat Dublin's 1-5ms.	Compete on signal quality + last-10s timing, not raw round-trips. Win-rate drift detector halts on regime change.
06	MAJOR	**Settlement migration USDC.e → native USDC** Mid-trade migration could leave positions inconsistent.	Phase 8 explicit migration runbook; double-write/read during rollout.
07	MAJOR	**Oracle stuck / Polygon reorg** Stale oracle while bot has open position.	Oracle staleness > 90s halts entries. Reorg: re-fetch from chain, accept ground truth.
08	MAJOR	**Schema/refusal failures from LLM** Sonnet meta could return malformed JSON or safety-refuse.	validate_or_retry wrapper on every call; <2 valid agents → Tier-3 fires.
09	MAJOR	**Bankroll reconciliation drift** Internal state ≠ on-chain USDC after partial fill / settled position.	Reconciler kill-switch: drift > $0.50 sustained 60s halts entries.
10	MINOR	**Min-order math** Polymarket 5-share / $4.75 min must always be coverable.	$5 flat for first 100 trades (clears minimum). At $1k bankroll this is 0.5% sizing — fine. Kelly only after empirical win-rate exists.
11	MINOR	**Anthropic Tier 1 RPM limits** $50 credit account is Tier 1 until $40 spent + 7 days.	Phase 3 fits in Tier 1; top up to $150 (Tier 2) before Phase 4 starts.
12	MINOR	**Tax / regulatory tracking** Per-trade tax basis at scale.	Manual CSV export through Phase 7; integrate Koinly at first profitable quarter.

Cost & Timeline

Cumulative spend per phase

Phase	Wall-clock	Phase USD cost	Cumulative	What drives the cost
0 — Foundations	1 day	$0	$0	—
1 — Wallet & Auth	0.5 day	$5	$5	Polygon gas
2 — Data Plane	3d + 7d capture	$5	$10	Alchemy free tier covers most
3 — Backtest + A/B	3-4 days	$15-50	$25-60	Anthropic API (cache-disciplined)
4 — Paper Trading	14 calendar days	$50-195	$75-255	Anthropic 24/7 (top up to $150)
5 — VPS Provisioning	0.5d + 24h staging	$5/mo	$80-260	Lightsail Dublin
6 — Live	14 calendar days	$50-195 + $1,000 capital	$1,130-1,455	Anthropic 24/7 (top up to $300) + at-risk capital
7 — Scale (per rung)	7d per rung minimum	bankroll-driven	+capital	Larger sums at risk; ops costs flat
8 — Hardening	ongoing	TBD	TBD	Per-item cost; ongoing OPEX

Sustained ops: ~$200/month budget driven primarily by Anthropic API at empirical $0.19-0.58/hour × 24/7. Optional reductions: cache discipline (system prompt as cache breakpoint), agent count reduction, lower-frequency meta-decider.

Operating Discipline (1/2)

25 anti-drift rules — pin above the desk

Session discipline

One Claude Code session = one phase task. End the session when the task ends.
Long sessions accumulate context rot; decisions get rationalized backwards.

Start each session by pasting the next task verbatim from RUNBOOK.md as the first prompt.
Forces you to know what you're starting.

Stop at 50% context use if no gate yet. Don't /compact mid-task.
Compaction summarizes Claude's perceived state — drift point.

Cap focused session length at 4 hours.
Tired operators accept first proposals.

End every session with a 2-line note in state/sessions/.md.
Future-you and future-Claude both need this anchor.

Decision logging

Code change → git commit, conventional-commits format. No WIP commits.
Small named commits make rollback surgical.

Architecture change → memory file edit + commit in same PR.
Code without preserved rationale dies on resumption.

Bug → GitHub issue with reproduction, even if fixed in the same hour.
Bug history is your taxonomy; recurrences signal a pattern.

"Maybe later" → GitHub issue with `icebox` label, never a TODO in code.
TODOs grow silently; icebox is visible and prunable.

Manual checkpoints

Read every diff Claude proposes before accepting. Never blanket-accept.
Drift happens in small unreviewed edits.

Run the gate command yourself in your own terminal. Watch it exit 0.
Claude can hallucinate gate output; your terminal cannot.

Tag green gates by hand: git tag phase-N-pass && git push --tags.
Tags are immovable rollback points.

Before advancing a phase, re-read the previous checkpoint memory file.
Gates exist to break momentum.

Operating Discipline (2/2)

macOS tooling

iTerm2 + tmux, one named tmux session per project. Windows: bot, repl, gates, git.
Separates concerns visually; survives terminal close.

VS Code (or Cursor) for diff review with GitLens, ruff, mypy. Even for code Claude wrote.
Editor view is more honest than chat view.

uv for Python on Apple Silicon. Skip system Python, skip pyenv, skip Anaconda.
Apple Silicon + scientific Python has a long tail of arch pain; uv handles it.

1Password CLI (or pass) for secrets. Use `op run -- python script.py` to inject at runtime.
Secrets never live on disk in plaintext.

Two browser profiles: one for Polymarket / Anthropic / GitHub, one for personal.
Scoped session cookies prevent cross-pollination of trading auth.

Anti-drift triggers

Same bug, two iterations → stop and write runbooks/incidents/-.md before iteration #3.
That's the 3-fix rule. Writing the incident usually surfaces the real root cause.

src/ changed but no design/ change in the same commit → reject the commit.
Silent design drift is the slowest, deadliest kind.

Gate criteria softened ("close enough", "flaky test") without an approved decision doc → revert, escalate.
Once you soften one gate, you'll soften them all.

Any "let me just" or "I'll quickly" from Claude → freeze the conversation, ask why.
Those phrases precede out-of-scope edits.

Recovery

When drift suspected: git log --oneline -20 first. Understand state before acting.
Panic-rollback loses real work.

Hard-reset to last green gate is your actual reset button: git reset --hard phase-N-pass.
Anything between gates is by definition unverified.

Post-recovery: write runbooks/postmortems/drift-.md (what drifted, when noticed, what reset to).
Drift patterns repeat; the postmortem is how you spot the pattern.

Sources & References

Verified canonical links

Polymarket — Official

Polymarket Developer Docs	docs.polymarket.com/developers/
py-clob-client (Python, primary)	github.com/Polymarket/py-clob-client
rs-clob-client (Rust)	github.com/Polymarket/rs-clob-client
agent-skills (drop into Claude)	github.com/Polymarket/agent-skills
agents (TS autonomous framework)	github.com/Polymarket/agents
poly-market-maker (reference MM)	github.com/Polymarket/poly-market-maker
Authentication (L1 + L2)	docs.polymarket.com/developers/CLOB/authentication
Maker Rebates Program	help.polymarket.com/en/articles/13364471

Reference bot implementations

Archetapp 5m bot gist	gist.github.com/Archetapp/7680adabc48f812a561ca79d73cbac69
aulekator 15m bot	github.com/aulekator/Polymarket-BTC-15-Minute-Trading-Bot
ThinkEnigmatic 5m arena	github.com/ThinkEnigmatic/polymarket-bot-arena
evan-kolberg backtest adapters	github.com/evan-kolberg/prediction-market-backtesting

Infrastructure & oracle

NautilusTrader Polymarket integration	nautilustrader.io/docs/latest/integrations/polymarket/
QuantVPS — Polymarket server location	quantvps.com/blog/polymarket-servers-location
Chainlink BTC/USD on Polygon	data.chain.link/feeds/polygon/mainnet/btc-usd
USDC.e to native USDC migration (Block)	theblock.co/post/388670

Claude tooling

iansinnott/obsidian-claude-code-mcp	github.com/iansinnott/obsidian-claude-code-mcp
MarkusPfundstein/mcp-obsidian	github.com/MarkusPfundstein/mcp-obsidian
caiovicentino/polymarket-mcp-server	github.com/caiovicentino/polymarket-mcp-server
artvandelay/polymarket-agents	github.com/artvandelay/polymarket-agents
agent-next/polymarket-paper-trader	github.com/agent-next/polymarket-paper-trader

---

*This document is a living specification. The companion files **RUNBOOK.md** (execution source-of-truth) and the locked-decisions memory file win when in conflict. Update at every phase gate.*